Bookory Project – Beginner-Friendly Walk-through

This project is a full-stack web application built with **ASP.NET Core MVC**. It implements an online bookstore called **Bookory**. The application allows customers to browse books, add them to a shopping cart, checkout, and pay for their orders. Administrators can manage books, view orders and customers, and see dashboard statistics. Below is a breakdown of every folder and file, along with an explanation of how they fit together.

1. Top-level overview

At the root of the Bookory-DEV1 folder you'll see two important items:

Item	Description
BookStoreMVC.sln	The solution file that Visual Studio uses to organise the project. It groups multiple projects if needed. Here it contains a single web project.
BookStoreMVC.csproj	The project file describing dependencies, target framework, and output settings. This project targets .NET 8.0 and references packages like *Microsoft.EntityFrameworkCore* for database access.

The remainder of the folder holds sub-directories. Below we explore each one.

2. Controllers folder

Controllers are classes that handle incoming HTTP requests. In ASP.NET MVC, they decide what to do when someone visits a particular URL and which view to show back to the user. Each controller corresponds to a specific area of the application.

File	Purpose
AccountController.cs	Handles user registration, login, logout, profile viewing/editing, and password hash/verification. It uses cookie authentication to store user sessions. For example, the Register action creates a new User record in the database, hashes their password for security, and logs them in.
AdminController.cs	Only accessible to users with the **ADMIN** role. It shows an admin dashboard with overall statistics such as total users, books, orders and sales. It also lists customers, in-stock books and out-of-stock books. A special [Authorize(Roles = "ADMIN")]

File	Purpose
	attribute protects this controller so normal users cannot access it.
BooksController.cs	Handles browsing, viewing, creating, editing and deleting books. For customers it shows a paginated list with category filters; administrators can add new books and manage existing ones. Image uploads are processed using IFormFile and saved into the wwwroot/images folder.
CartController.cs	Manages the shopping cart. Users must be logged in to access it ([Authorize]). Actions include viewing cart items, adding to cart, updating quantities, removing items, checking out, paying, and displaying a success page. It uses helper view models (CheckoutVm and PaymentVm) to collect shipping and payment information from forms.
HomeController.cs	Serves general pages like Home, Privacy, About Us and Contact Us. The Index action fetches a few books from the database to display on the homepage.
OrdersController.cs	Displays orders and order details. Admins can view all orders and change their status to delivered or cancelled. Customers see only their own orders. It also supports requesting returns and cancelling orders.

How controllers work together

- **Routing** – When a user goes to https://yourdomain.com/Books, the framework looks up the BooksController and runs its Index action. The method executes code (e.g., fetch books from the database via services) and returns a view.
- **Dependency injection** – Each controller is given (injected) the classes it needs: ApplicationDbContext (database context) or service interfaces (IBookService, ICartService, etc.). ASP.NET automatically creates these objects based on the registrations in Program.cs. This pattern simplifies testing and separation of concerns.
- **Authorization** – Attributes like [Authorize] or [Authorize(Roles = "ADMIN")] restrict access to authenticated users or specific roles. The AccountController sets up user roles and cookies so the framework knows who is logged in.

3. Models folder

Models represent the data in the application. They mirror database tables and are used throughout the application.

Model	Fields (simplified)	Description
User	Id, Username, PasswordHash, Role, Email, FirstName, LastName, DateOfBirth, Gender, Phone, Address	Represents a customer or administrator. The **Role** determines whether the user has admin rights. Passwords are stored as salted hashes to prevent reading the plain text password.
Category	Id, Name	A book category like Fiction, Science or Technology. One category can have many books (one-to-many relationship).
Book	Id, Title, Description, Price, CategoryId, StockQuantity, ImageUrl	Represents a book listing. CategoryId links back to a Category. StockQuantity tracks how many are available.
CartItem	Id, UserId, BookId, Quantity	Holds the books a user has added to their cart. A single user can have many cart items.
Order	Id, UserId, TotalAmount, OrderDate, Status, ShippingName, ShippingAddress, etc.	Represents a completed checkout. It records the user, the order total, shipping info and a status (pending, shipped, delivered, cancelled, return requested, etc.).
OrderItem	Id, OrderId, BookId, Quantity, UnitPrice	Line items within an order, connecting a book to the order along with quantity and price.
Payment	Id, OrderId, Amount, PaymentStatus, PaymentMethod, PaymentDate	Tracks payment information for an order. PaymentStatus can be pending, completed or failed. Payment method is stored as a string (UPI, Card, COD, etc.).
PaginatedList	–	A generic helper for paging

Model	Fields (simplified)	Description
		large lists. It calculates the total number of pages, the current page and whether previous/next buttons should be enabled.
DbInitializer	–	Contains a static Seed method that inserts initial data (categories, sample books and an admin user). It uses a PBKDF2 function to hash the default admin password. This method runs once at startup inside Program.cs to ensure the database has content.
ApplicationDbContext	–	Inherits from DbContext (Entity Framework Core). It exposes DbSet<T> properties for each model class so EF can map them to tables. It also configures relationships (e.g., a cart item belongs to one user and one book) and converts enums like PaymentStatus to strings in the database.

Enumerations

Several enums make code more readable:

- **UserRole** – CUSTOMER or ADMIN.
- **Gender** – MALE, FEMALE, OTHER.
- **OrderStatus** – covers PENDING, SHIPPED, DELIVERED, CANCELLED, plus return statuses.
- **PaymentStatus** – PENDING, COMPLETED, FAILED.

Storing these as enums means you get compile-time safety (you can't accidentally assign an invalid status) and easier maintenance.

4. Services folder

Services encapsulate business logic separate from controllers. They hide details of how data is stored (e.g. using Entity Framework) and provide clean methods that controllers can call.

Book service

- **IBookService** – defines the methods for listing categories, retrieving books (optionally filtered by category), getting a book by ID, adding, updating and deleting books.
- **BookService** – implements the interface using ApplicationDbContext. It uses asynchronous EF methods like ToListAsync() and FirstOrDefaultAsync() to avoid blocking the server.

Cart service

- **ICartService** – defines methods for managing cart items: getting the current user's cart, adding to cart, updating quantity, removing an item and clearing the cart.
- **CartService** – implements ICartService. It checks stock levels before adding more items, clamps the quantity to available stock, and ensures changes persist in the database.

Order service

- **IOrderService** – defines methods for getting a list of orders (filtered by user role), retrieving a single order, creating a new order from the cart, and updating the order status.
- **OrderService** – implements IOrderService. Its CreateOrderAsync method is noteworthy: it wraps the creation in a **database transaction**. This means if any part fails (e.g. not enough stock), the whole operation is rolled back. It reduces stock quantities accordingly and clears the cart once the order is created. Updating the order status also updates related Payment records when appropriate.

Payment service

- **IPaymentService** – defines a single method ProcessPaymentAsync, which takes the user ID, order ID, amount, payment method and success flag.
- **PaymentService** – implements payment processing logic. It verifies that the order belongs to the user, determines how the payment and order statuses should change depending on the method (e.g. COD vs. online) and whether payment succeeded, creates a Payment record and updates the order status. For online payments it marks orders as pending or cancelled based on the success flag. For cash on delivery it leaves the payment pending until delivery.

Why services?

Without services, controllers would directly interact with ApplicationDbContext, leading to bulky code and tight coupling. By introducing interfaces, the project gains flexibility: you can swap implementations, write unit tests with mocks, and keep controllers focused on request handling instead of data access details.

5. Views folder

Views are Razor Pages (.cshtml) that generate HTML to display in the browser. They use a combination of HTML, C# code and special Razor syntax (@Model, @foreach, etc.). Views are

organised into sub-folders matching controller names. Each action typically has a corresponding view.

Shared layout files

- **Shared/_Layout.cshtml** – The base HTML template used by most pages. It defines the common navbar, footer, imports CSS/JS (Bootstrap, icons) and calls @RenderBody() where individual views insert their content. It also includes navigation links that change depending on whether a user is logged in or an admin.
- **Shared/_AdminLayout.cshtml** – An alternate layout for admin pages. It provides an admin-specific menu and uses a separate stylesheet admindashboard.css.
- **Shared/_ValidationScriptsPartial.cshtml** – Contains script references for client-side form validation (e.g. jQuery validation). This is automatically included in form views that use <partial> tag helpers.

Account views

- **Register.cshtml** – A registration form that collects username, email, password, confirm password, first name, last name, date of birth, gender, phone and address. The form posts back to AccountController.Register.
- **Login.cshtml** – A login form for existing users. On success the controller checks the role and redirects the user to the appropriate page (home or admin dashboard).
- **Profile.cshtml / EditProfile.cshtml** – Show and edit a user's personal details. These views bind to the User model and use hidden fields for things that should not be changed (e.g. ID, role).
- **ForgotPassword.cshtml** – A simple form that collects an email address. In a real application you would generate a password reset token and email it; here it just displays a message.

Books views

- **Index.cshtml** – Shows a paginated grid of book cards with filtering. It uses the PaginatedList model to render previous/next buttons and indicates the current page. Admin users have additional "Edit" and "Delete" buttons on each card. A price range slider and search box filter the visible books on the current page.
- **Create.cshtml, Edit.cshtml, Delete.cshtml** – Admin-only forms for adding, editing and deleting books. They allow uploading an image file and selecting a category.
- **Details.cshtml** – Shows a single book with its description, category, price and available stock. Customers can add the book to their cart if it's in stock.

Cart views

- **Index.cshtml** – Lists the books in the user's cart. Each row displays the title, unit price, quantity with a drop-down to change quantity, and a remove button.
- **Checkout.cshtml** – Form collecting shipping details (name, address, city, state, zip, phone). Once submitted, it calls the Checkout POST action which creates an order.
- **Payment.cshtml** – Shows a summary of the order and a form to choose payment method. There are radio buttons for UPI, card and cash on delivery. Card fields

(number, expiry, CVV, cardholder name) and UPI ID fields appear conditionally. You can simulate success or failure via the Success field in PaymentVm.

- **Success.cshtml** – Displays a success or failure message after payment, along with order details like items and total amount.

Orders views

- **Index.cshtml** – Shows a table of orders. Customers see only their orders; administrators see all orders. Columns include order date, total, status, payment status and actions (view details, update status for admins).
- **Details.cshtml** – Shows a single order's items, shipping info and status. Customers can request a return or cancel the order if certain conditions are met. Admins have a drop-down to update the status (e.g. mark as delivered). When a delivered order is returned to stock or cancelled, the code in OrdersController adjusts book stock levels accordingly.

Admin views

- **Dashboard.cshtml** – A summary page showing total users, total books, total orders, in-stock and out-of-stock counts, and total sales. It lists the five most recent orders with their user names, amounts and statuses.
- **Users.cshtml** – A table listing all customers (non-admin users) with their names, email and registration details.
- **Instock.cshtml / Outofstock.cshtml** – Lists books that are currently available or out of stock.

Home views

- **Index.cshtml** – A simple landing page that displays a few highlighted books. It links to the shop (Books page).
- **AboutUs.cshtml** – Static content describing the bookstore. It uses the standard layout and includes sections like mission statement, history or team.
- **ContactUs.cshtml** – A form for users to send a message to the store, plus contact information. In this project the form is static (no back-end handling).
- **Error.cshtml** – An error page displayed when unhandled exceptions occur.
- **Privacy.cshtml** – A placeholder privacy policy page.

This structure shows how views correspond to each controller action, making it easy to navigate the user interface.

6. wwwroot folder

The wwwroot folder holds all static files served directly by the web server:

- **css/site.css** – Custom styling to complement the Bootstrap framework. It includes classes for the product cards, buttons, layout spacing and admin dashboard.
- **images/** – A collection of images used across the site. Examples include brief_history_of_time.jpg, clean_code.jpg, great_gatsby.jpg (book covers), login-bg.png for the login page background, checkout-bg.png for the checkout page, and

icons like logo-bookory.svg. When an administrator uploads a new book image, it's saved into this folder.

Keeping static assets under wwwroot allows the ASP.NET pipeline to serve them efficiently without passing through the MVC layer.

7. Other important files

Program.cs

Program.cs is the **entry point** of the web application. It performs the following tasks:

1. **Creates a WebApplicationBuilder** – This constructs the application and holds configuration settings from appsettings.json.
2. **Registers services** – It adds MVC controllers with views, registers IHttpContextAccessor to access the current user in views, configures the database context to use SQL Server via a connection string, and sets up authentication using cookies. It also configures an AdminOnly authorization policy and registers all the services (book, cart, order, payment) via dependency injection.
3. **Builds the application** – After configuration the builder creates the app object.
4. **Configures middleware** – It sets up exception handling, HTTPS redirection, serving of static files, routing, authentication and authorization. At the end it defines the default route pattern ({controller=Home}/{action=Index}/{id?}).
5. **Applies pending migrations and seeds initial data** – A scope is created, the database is migrated, and DbInitializer.Seed inserts categories, sample books and an admin user on first run.
6. **Runs the application** – app.Run() starts listening for HTTP requests.

appsettings.json

This file holds configuration such as the database connection string (named DefaultConnection). For local development it points to a SQL Server instance. In production you would update this with your actual database details.

bin and obj folders

These are build output and temporary files generated when the project is compiled. bin/Debug/net8.0 contains the compiled DLL (BookStoreMVC.dll) and satellite assemblies for different languages. You rarely touch these folders directly; they are re-created on each build.

8. Workflow of the application

To understand how a typical request flows through the application, consider the scenario of a customer **buying a book**:

1. **Browsing** – The user visits /Books. The MVC pipeline routes this to BooksController.Index. The controller asks BookService for a list of books (optionally

filtered by category), wraps them in a PaginatedList, and returns the Index.cshtml view. The view displays the books, category filters and pagination links.

2. **Adding to cart** – When the user clicks "Add to Cart" on a book card, the browser sends a GET request to /Cart/Add?bookId=…. The CartController.Add action calls CartService.AddToCartAsync which checks stock and either creates or increments a CartItem. The user is redirected to the cart page or back to the book list depending on a query parameter.
3. **Viewing cart** – /Cart triggers CartController.Index. It retrieves all cart items for the current user via ICartService and passes them to the view.
4. **Checkout** – At /Cart/Checkout (GET), the cart items are summarised; shipping details are collected via a form. Submitting the form (POST) calls CartController.Checkout, which uses IOrderService.CreateOrderAsync to create an Order with OrderItems, deducts stock and empties the cart. If successful it redirects to the payment page.
5. **Payment** – /Cart/Payment?orderId=… shows a form that collects payment details (PaymentVm). Submitting the form (POST) calls CartController.Payment. It uses IPaymentService.ProcessPaymentAsync which validates that the order belongs to the user, determines new statuses based on payment method and success flag, creates a Payment record and updates the order. Then it redirects to /Cart/Success.
6. **Success page** – The Success.cshtml view displays a confirmation or error message after payment, along with order details. The user can now go to their orders page.
7. **Order history** – /Orders displays all orders for the user. Clicking "Details" shows OrdersController.Details, which loads the order, its items, payment, user, and category details. From here the user can cancel or request a return if the order is eligible.

Administration flows follow similar patterns but use AdminController and admin-specific views and menus. Admins can create or delete books via BooksController, see recent orders in the dashboard and update order statuses.

9. How to explain this project in an interview

When discussing this project, it helps to structure your explanation around the architecture, features, design patterns and security considerations:

- **Architecture** – Mention that this is an **ASP.NET Core MVC** application following the **Model-View-Controller** pattern. Models represent the data and database tables (EF Core), controllers handle HTTP requests and coordinate services, and views render HTML using Razor. Services encapsulate business logic and data access, making the controllers thinner and easier to maintain.
- **Key features** – Explain that customers can register, log in, browse and filter books, add items to a cart, go through a multi-step checkout, pay using different methods (simulate UPI, card or cash on delivery), view past orders, cancel or request returns, and edit their profiles. Admins can manage books (CRUD), see dashboards with statistics, manage customer data and update order statuses. The site uses Bootstrap for responsive layout and includes client-side scripts for filtering and dynamic forms.

- **Data access** – Describe using **Entity Framework Core** with SQL Server. The ApplicationDbContext defines DbSet<T> properties for each table and configures relationships. Asynchronous methods (await/async) allow the server to handle other requests while waiting for database operations, improving scalability.
- **Authentication & Authorization** – Point out that the project uses cookie authentication and the [Authorize] attribute to protect pages. Roles (UserRole) differentiate admins from customers. Passwords are hashed using PBKDF2 with random salt; DbInitializer seeds an admin user with a hashed default password. Claim-based identity is used to retrieve the current user in controllers and views.
- **Transaction management** – Highlight that the order creation process uses a **database transaction** to ensure consistency: if stock is insufficient, the order is not created and stock quantities remain unchanged. Payment processing also checks that the order belongs to the current user and sets statuses accordingly.
- **User experience** – Explain how pagination improves performance on the books list, how client-side scripts filter items without reloading the page and how forms are validated with data annotations. Discuss the admin dashboard providing quick insights into the system.
- **Extensibility** – You could mention how the service interfaces make the codebase extensible. For example, you could replace IPaymentService with a real payment gateway integration or swap the database provider by changing UseSqlServer to UseSqlite without rewriting business logic.

By covering these points you demonstrate understanding of both the technical details and the reasoning behind them.

10. Summary

This **Bookory** project is a comprehensive but approachable example of building a full-stack web application with **ASP.NET Core MVC**. It covers essential web concepts—routing, views, controllers, dependency injection, authentication/authorization, data modelling and persistence, transactions, and user experience enhancements like pagination and role-based menus. Everything from the folder structure to the services and Razor views is designed to follow clean architecture principles, making it an excellent learning resource for beginners and a solid showcase for interviews.